IFT163 HOMEWORK 2: World Models and Model-Based Planning

Thuan Nguyen

March 3, 2026

My HuggingFace account is **NMThuan032k**. I have submitted my code on the leaderboad on HuggingFace.

1 Simple World Model Planning

Goal: Implement and train a simple pose-based world model that predicts the next robot pose given the current pose and action.

1.1 Understanding the SimpleWorldModel

The SimpleWorldModel operates as a deterministic engine to predict the physical dynamics of the environment. In particular, it uses a simple MLP to predict the next normalized pose ($p_{t+1} \in \mathbb{R}^7$) and reward ($r_t \in \mathbb{R}$) given that input is the normalized pose ($p_t \in \mathbb{R}^7$) and the normalized action ($a_t \in \mathbb{R}^7$).

Note: For the question 1 and 2, 'pose' is considered as 'state'. I will alternate 'pose' and 'state' because they have the same meaning.

1.2 Training the Simple World Model

Firstly, I will provide a formula for the SimpleWorldModel, as follows:

$$\hat{p}_{t+1}, \hat{r}_t = f_\theta(p_t, a_t)$$

For the loss term, I chose MSE for both pose loss and reward loss. In particular,

- **Pose loss:**

$$\mathcal{L}_{pose} = \frac{1}{N} \sum (\hat{p}_{t+1} - p_{t+1})^2$$

  Pose is continuous and low-dimensional, so MSE is a good choice. MSE is easy to optimize and gives stable gradients. MSE is stable as long as values are scaled so we need to normalize the inputs.

- **Reward loss:**

$$\mathcal{L}_{reward} = \frac{1}{N} \sum (\hat{r}_t - r_t)^2$$

  The target reward is a scalar value. I chose MSE simply because it easy to optimize and gives stable gradients.

- **Total loss** is just the unweighted sum of these two components.
- The model architecture uses LayerNorm + SiLU residual MLP blocks, which reduces gradient explosion risk.
- In order to normalize pose and action, I use the predefined encode_pose and encode_action methods inherited from the class GRPBase.

1.3 Implementing CEM Planning

- I initialize the action distribution as an independent Gaussian over action sequences, with mean at 0 and std at 6. The zero mean matches our overall convention that actions are normalized, so 0 is a sensible "do-nothing / average-action" prior. The std at 6 because I investigated the range of encoded action is from -6 to 6.
- I keep a fixed number of CEM iterations, which is 20, because I want CEM to converge. If the number of CEM iterations is too small, it will not be guaranteed to converge. If the number of CEM iterations is too large, it will be time-wasted.

1.4 Questions for part 1

- Loss curves (pose loss and reward loss over epochs) Figure 1 illustrates the training loss curves

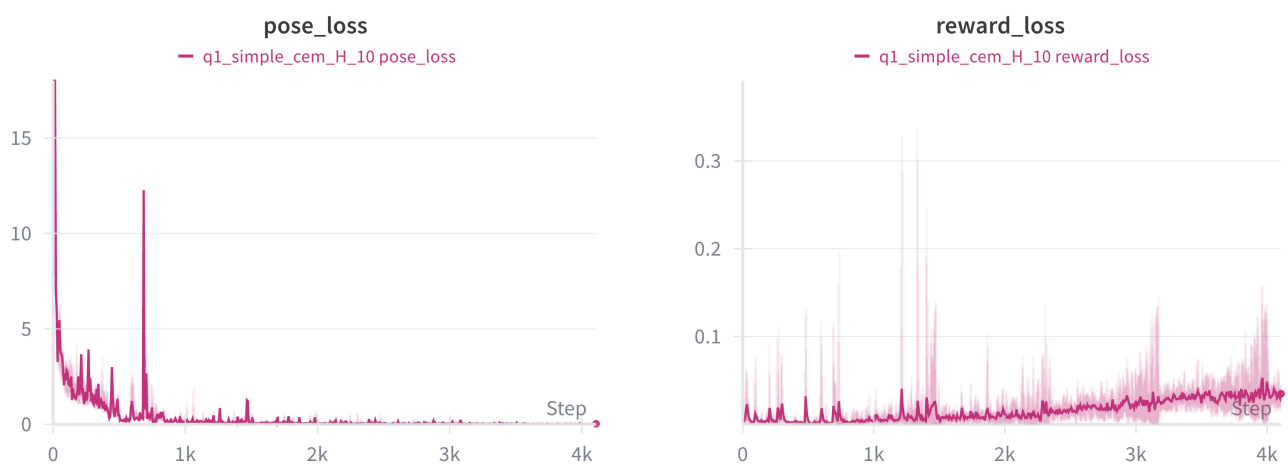

Figure 1: Loss curves (pose loss (left) and reward loss (right) over epochs)

including poss loss and reward loss of SimpleWorldModel. The pose loss quickly decreases and converges to zero, which shows that the SimpleWorldModel learned the robot transition patterns. Early spikes occur due to rare states, but they disappear after sufficient training. In contrast, the reward loss stays noisy and slowly goes up because it's much harder to predict rewards. I found that most dataset states have zero rewards, while only goal states provide non-zero values. In addition, the rewards are updated continuously through environmental interaction during training (after every 50 epochs) and evaluation. These target changes could make MSE increase but make planning more useful with dense rewards returned from the environment. Another insight: the model learns physical transitions well, but it doesn't

always predict goals correctly. This means CEM planning should work, but it will be limited by reward estimation accuracy, which impacts evaluating and ranking in CEM.

- Success rate vs planning horizon plot (horizon = 5, 10, 15)

Figure 2: Impact of planning horizon on success rate and reward

- Success rate vs number of CEM samples (samples = 50, 100, 200)
- **Discussion:** How does planning horizon affect performance? What is the trade-off between sample count and computation time? **Answer:** Figure 2 illustrates how the planning horizon affects the success rate and reward. As you can see, the planning horizon increases from 5 to 20, and the average reward improves. This indicates that a longer lookahead enables better long-term planning. However, increasing the planning horizon to 25 may result in a reduction in reward. This case is likely due to accumulated model prediction errors, including pose prediction errors and reward prediction errors. Hence, the planning horizon, which controls how far ahead the robot can imagine, can improve foresight but may suffer from model error accumulation if it is longer.

  Figure 3 illustrates how the number of samples affects the success rate and reward. As you can see, a greater number of planning samples can generally improve average reward, as greater sampled action sequences increase the probability of finding higher-return trajectories. However, this improvement comes at the cost of increased computation time, since CEM must evaluate every sampled sequence through the world model. Planning complexity scales approximately linearly with both the number of samples and the planning horizon. Therefore, there is a clear trade-off: higher sample counts improve planning quality but significantly increase inference time, making real-time deployment more challenging.

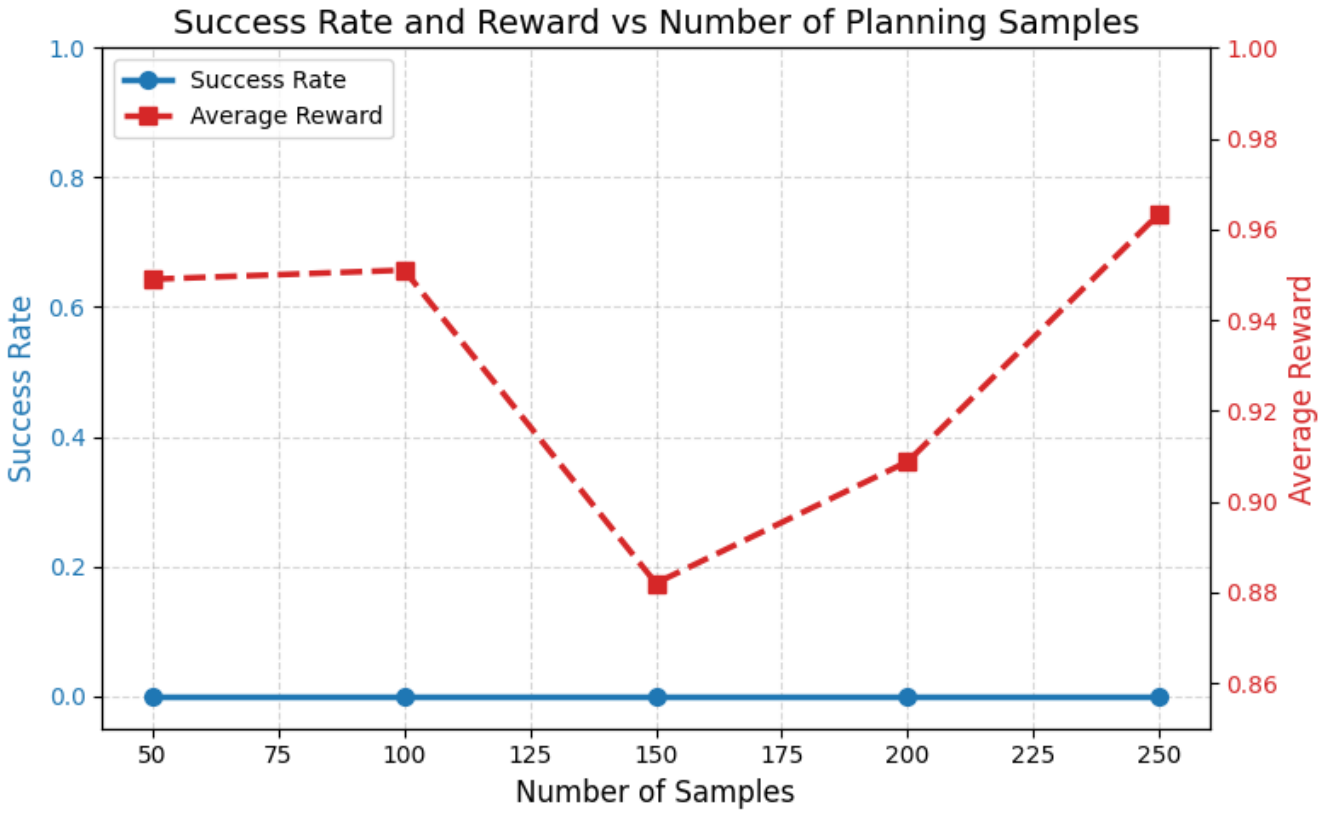

Figure 3: Impact of number of samples on success rate and reward

2 Policy-Guided Planning

Goal: Improve CEM planning efficiency by using a learned policy to initialise the action distribution.

2.1 Understanding Policy Initialization

Before implementing the CEM loop, we first predict an action sequence distribution given the current state. I implement a loop to create this action sequence by using both the policy model and the world model. In particular, given the current state, I first use the policy model to predict an action distribution, then feed both the current state and the action mean to the world model to predict the next state. I do it repeatedly.

2.2 Implementing Policy Training

- Batching and normalizing the data: The shape of poses and actions are $(N, T, pose_dim)$ and, $(N, T, action_dim)$ respectively. For behavior cloning, the policy just needs supervised pairs (s_t, a_t), so I flatten time $(N, T, *) \rightarrow (N \cdot T, *)$. Then I train with mini-batches of size B sampled from thoes flattened pairs. Flattening treats each timestep as an independent supervised sample, which matches how the policy is used at test time (one-step inference). Mini-batches give stable gradients and allow me scale training without storing huge rollouts. In terms of normalizing, I used the pre-defined encode_action and encode_pose methods

- **Training loss:** Recall that we are working in the normalized space, which means that the expert actions are normalised. The policy is trained to output a continuous action distribution rather than a single deterministic action. Therefore, the network outputs a tensor (14-D) concatenating a mean (μ) and a log standard deviation ($log\sigma$). As I mentioned above, the encoded action range is [-6,6], so I clamp ($log\sigma$) to a range $[-5, 2]$. So the policy is

$$\pi(a_t|s_t, \theta) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$$

  I selected NLL instead of MSE due to the potential noise in human or expert demonstrations. In addition, if the expert occasionally navigates left and at other times right around an obstacle, MSE would compute the average of the two paths, resulting in the robot colliding directly with the obstacle.

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s,a)\sim\mathcal{D}}\big[\log \pi_\theta(a \mid s)\big]$$

- **Sanity check/diagnostic metric.** To verify the policy was acquired independently of the NLL loss, a diagnostic metric "MSE" was monitored in the returned dictionary. I computed MSE between the predicted action mean and the expert action to verify deterministic alignment.

$$\text{MSE} = \|a - \mu_\theta(s)\|^2$$

2.3 Integrating Trained policy with CEM

- **How to initialize action sequence distribution before the first CEM iteration?** My policy outputs a Gaussian distribution of action per timestep. Using the world model, I can roll out the policy for H steps to build a full initial sequence:

  - $\mu^\pi_{0:H-1}$: the mean action sequence predicted by the policy rollout
  - $\sigma^\pi_{0:H-1}$: the policy's predicted per-dimension std (or a fixed one)

  This initialization directly anchors CEM around a plan that's likely to be reasonable given training data which can help to converge faster than a hardcoded standard normal distribution.

- **Fixed hyperparameters and Exploration vs. Exploitation**: I fixed the temperature at 0.3 to ensure that it can explore and exploit. The temperature is too low (no exploration) or too high (too random). In addition, I clamp the policy output's standard deviation at min=1e-3 to avoid the overconfidence of policy. This makes it always explore.

2.4 Questions for part 2

Method	Success Rate	Avg Reward	Avg Planning Time
Random CEM (100 samples)	0	0.929	18.31s
Policy-guided CEM (50)	0	1.103	19.31s
Policy alone (no CEM)	0	0.77	0.08s

Table 1: Success rate comparison of different planning methods.

- How much does the policy improve CEM efficiency? Compare success rate with 50 samples (policy-guided) vs 100 samples (random) **Answer:** Although both methods achieved a 0 success rate, policy-guided CEM shows an efficiency signal in the optimization objective because the average reward increase from 0.929 (random CEM, 100 samples) to 1.103 (policy-guided CEM, 50 samples). However, policy-guided CEM takes longer time than random CEM (19.31s compared to 18.31s). This implies the policy-guided version likely introduces extra overhead caused by policy model inference, so "fewer samples" did not translate into lower wall-clock time. This is a trade-off.

- Can the policy alone (without CEM refinement) solve the task? Why or why not? **Answer:** No, as the results are illustrated in table 1. The probable reason is that the policy is trained from limited and imperfect offline trajectories and encounters a distribution shift at the test time. This leads to the accumulative errors that make the task fail, especially in the manipulation tasks.

- What are the benefits of combining learned policies with model-based planning? **Answer:** Although success is not achieved, Table 1 shows some benefits:

  1. Policy provides a good prior for CEM.
  2. CEM planning can correct mistakes from imperfect policy actions by searching locally around the policy's proposal.

- Provide one plot or table that demonstrates a non-obvious failure mode of the policy or planner.

3 DreamerV3 World Model

Goal: Replace the simple pose-based model with DreamerV3, a sophisticated world model that works with image observations.

3.1 Training DreamerV3

All losses are computed inside DreamerV3.compute_loss(...) and combined as:

$$\mathcal{L}_{total} = \lambda_{pred} * \mathcal{L}_{pred} + \lambda_{dyn} * \mathcal{L}_{dyn} + \lambda_{rep} * \mathcal{L}_{rep}$$

where:

$$\mathcal{L}_{pred} = \mathcal{L}_{recon} + \mathcal{L}_{reward} + \mathcal{L}_{continue}$$

and each term is:

1. **Reconstruction loss** I implemented it as pixel MSE in normalized image space. This is a straightforward reconstruction objective; it encourages the latent state to preserve enough information to decode the observation.

2. **Reward loss** I implemented it as MSE on symlog reward. This is a common Dreamer trick. Symlog compresses large magnitudes and improve numeric stability for reward/value learning.

3. **Continue prediction loss** I implemented it as binary cross entropy with logits. This teaches termination dynamics (useful for imagined rollouts to not overcount rewards past terminal states).

4. **Dynamics+representation losses** I implemented them as two KL losses between posterior and prior categorical latents, as follows:

$$\mathcal{L}_{\text{dyn}}(\phi) := \max(1,\ \text{KL}[\text{sg}(q_\phi(z_t \mid h_t, x_t)) \parallel p_\phi(z_t \mid h_t)])$$

$$\mathcal{L}_{\text{rep}}(\phi) := \max(1,\ \text{KL}[q_\phi(z_t \mid h_t, x_t) \parallel \text{sg}(p_\phi(z_t \mid h_t))])$$

   I also apply a free-nats floor to prevent KL collapse and helps keep a minimum information flow in the latent.

5. I utilised the default weights to calculate the total loss. This achieves a balance between acquiring an informative latent representation, preventing posterior collapse, and maintaining the model's predictive capabilities for planning.

3.2 CEM Planning with DreamerV3

- **How imagined rollouts are generated.**

  1. **Start from an inferred latent state:** Before planning, DreamerV3 constructs a compact latent state from real observations using the encoder and RSSM posterior updates. Concretely, CEMPlanner._forward_dreamer(...) encodes the observation sequence and unrolls the RSSM to produce a final latent $state = 'h', ' z', ...$ which is passed into planner.

  2. **Sample candidate action sequences** is the same as the above question 1.

3. **Imagine forward using the RSSM prior (no observations)**. For each sampled action sequence, we "imagine" the future entirely in latent space using the RSSM transition (the prior) with embed=None, meaning no posterior correction.

- **How rewards are accumulated** At each imagined step, I construct a feature vector $\phi_t = [h_t, z_t]$ and predict reward using the reward head $\hat{r}_t = R_\theta(\phi_t)$. And then I compute the total predicted return $\hat{G}^{(k)}$ across all imagined step for each candidate sequence as a simple undiscounted sum. That scalar $\hat{G}^{(k)}$ is what CEM uses to rank sequences, select elites, and update (μ, σ).

- Numerical/stability issues

  1. KL/ categorical stability in the RSSM: Discrete latents can cause instability when some categories get a probability near 0 (KL can blow up). In my Dreamer implementation explicitly adds unimix smoothing inside DreamerV3.sample_stochastic(...). This avoids exact zeros and prevents infinite KL / degenerate distributions during training, which indirectly stabilizes imagination quality for planning.
  2. CEM distribution collapse: CEM can destabilize when elite variance becomes tiny (std $\to 0$), causing poor exploration or numerical issues. I clamps this std with a minimum at 10^{-3}. This prevents NaNs and keeps some exploration.

3.3 Running Experiments

3.4 Questions for Part 3

1. How does DreamerV3 compare to SimpleWorldModel in terms of:

   - Training time. **Answer:** DreamerV3 takes much longer to train than SimpleWorldModel because it has a bigger structure and needs to handle complex images using a CNN encoder and RSSM model, while SimpleWorldModel only predicts 7-D pose changes with a small MLP.
   - Sample efficiency. **Answer:** Dreamer V3 is more sample-efficient than the Simple World Model. The SimpleWorldModel only learns deterministic robot pose transitions and relies on CEM planning to search for rewarding actions, which effectively require many sampled trajectories to discover successful behaviours. In contrast, DreamerV3 learns a latent representation of the full environment from images using a recurrent state-space model. This latent state captures object positions, contact events, and task progress, allowing the agent to imagine future trajectories and learn from internal rollouts rather than only real transitions. As a result, DreamerV3 can extract more useful learning signals from the same dataset and generalize better, even though it is slower to train computationally.
   - Final success rate. **Answer:** Both models achieved a zero success rate.

2. Why might image-based models be beneficial even if pose is available? **Answer:** From my point of view, image-based models offer a more comprehensive representation of the environment compared to pose-based models. The 7-D end-effector pose solely depicts the robot's configuration, ignoring object positions, contact states, and the success of a grasp. Consequently, two distinct world states (e.g., grasping an object versus not grasping it) may exhibit

identical poses, complicating reward prediction. Conversely, images encompass task-relevant information, including object locations, end-effector openings, and collisions. DreamerV3 encodes images into a latent state that contains hidden variables. This lets the world model make better guesses about rewards and next states. Consequently, image-based models enhance planning by capturing the complete structure of the environment rather than solely the robot's motion, and they exhibit superior generalization for novel scenes or sim-to-real contexts.

3. Visualize image reconstructions - what features did the model learn?

4. Describe one implementation detail that you had to discover by reading the code rather than the writeup. **Answer:** I described it clearly above.

4 Image-Based Policy

4.1 Policy Architecture for Images

I utilise the pre-trained encoder and RSSM of DreamerV3 to generate inputs for the policy network, instead of constructing an independent CNN. My architectural choice is predicated on the fact that the DreamerV3 world model has been trained to encapsulate a compact and significant representation of the world from high-dimensional images through reconstruction and dynamic losses. By inputting $[h_t, z_t]$ into the policy, it acquires a profound temporal and spatial comprehension, significantly enhancing the sample efficiency of behaviour cloning.

4.2 Policy-Guided CEM with DreamerV3

Requirement: Describe how information flows from image to RSSM state to action distribution, and explain any assumptions you make about temporal alignment or horizon length.

Image → RSSM state → policy action distribution → CEM refinement

1. **Image to RSSM latent state.** Given an observation history including $(o_{0,t})$ and the corresponding previous actions $(a_{0,t})$, I compute the current RSSM state by first encoding each image with Dreamer encoder $e_t = DreamerV3.encoder(o_t)$ and then unrolling the RSSM with posterior updates $(h_t, z_t) = Dreamer.rssm_step(h_{t-1}, z_{t-1}, a_{t-1}, e_t)$.

2. **RSSM latent state to initial action distribution.** I designed the policy so it does not directly process images. Instead, it processes RSSM features computed above. $\phi = [h, z]$. To generate an initial distribution for action sequences of length (H), the policy is rolled out in imagination based on the DreamerV3 world model as described in part 2.

3. **CEM uses the world model to refine the initial action distribution.** I use the default CEM to refine the action distribution. For each sequence, I simulate rollouts using the RSSM step and accumulate the predicted rewards to rank. Then fitting the distribution to the top (M) elite sequences.

4. **Assumption:** We align actions such that the RSSM step that incorporates o_t uses a_{t-1}, with a dummy zero action at the first step. The reason behind this assumption is that this approach matches standard RSSM timing and avoids "peeking" with the wrong action.

4.3 Running Experiments

4.4 Questions for Part 4

1. Create a comparison table of all methods. Which performs best? **Answer:** Based on table 2, all methods achieved a zero success rate, so it is hard to identify which method performs best. However, I think that Dreamerv3 is large, so we need a big dataset to train it. Because our dataset is quite small, so even though Dreamer models are more expressive, they underperformed in reward and did not achieve success. In addition, reward prediction from images seems difficult.

2. What are the trade-offs between model complexity and performance? **Answer:** From my point of view, DreamerV3 models contain 16–18M parameters and process high-dimensional

Method	Success Rate	Avg Reward	Avg Planning Time	Model Params
Simple + Random CEM	0	0.929	18.31s	1M
Simple + Policy-guided CEM	0	1.103	19.31s	1M + 1,1M
DreamerV3 + Random CEM	0	0.698	15.33s	16.8M
DreamerV3 + Policy-guided CEM	0	0.729	16.5s	16,8M+1,6M

Table 2: A comparison of all methods.

image inputs. This makes them computationally expensive and harder to optimize. Table 1 suggests that the Dreamerv3 may require longer training or better tuning to get better results. Currently, I cannot identify trade-offs between model complexity and performance.

3. When would you use each approach in practice? **Answer:** SimpleWorldModel with CEM is good for simple physical tasks or debugging because it trains quickly and doesn't need much learning. However, it is slow at inference and can't reason about objects. Adding a policy (policy-guided CEM) is helpful in structured settings where things need to be done quickly and reliably because the policy gives the first step, and the planner corrects mistakes. DreamerV3 with CEM is good for tasks that require interacting with objects and changing environments because it learns how the visual world works. However, CEM planning with DreamerV3 consumes a lot of computing resources. Finally, DreamerV3 with policy-guided CEM is the best option out of these 4 for real-world use cases in complicated settings. It combines action policy, which gives the first step, with model-based correction. This makes it both strong and generalizable. Nonetheless, it needs more training and computing power.

4. How does sample efficiency compare between the methods?

5. Describe one result that surprised you and provide a plausible explanation.

References